Software and Web Security

6COSC019W- Cyber Security

Dr Mohemmed Sha Mohamed Kunju

March 02, 2026

Prepared by Dr Ayman El Hajjar

UNIVERSITY OF WESTMINSTER

Overview

1. Introduction to threats
2. Key web threats
3. Application Threats
4. Countermeasures

What Are Web Threats?

- Attacks on web apps—servers, browsers, or both.
- **Why care?** 80% of breaches exploit web flaws (2024 estimate).

Common Themes

- **Unvalidated Input**: Apps trust users too much.
- **Exploitable Code**: Injection, scripting flaws.

OWASP Top 10

- The OWASP Top 10 is a standard awareness document for developers and web application security about the most critical security risks to web applications.
  - A01 - Broken Access Control
  - A02 - Security Misconfiguration
  - A03 - Software Supply Chain Failures
  - A04 - Cryptographic Failures
  - A05 - Injection
  - A06 - Insecure Design
  - A07 - Authentication Failures
  - A08 - Software or Data Integrity Failures
  - A09 - Security Logging and Alerting Failures
  - A10 - Mishandling of Exceptional Conditions

- OWASP top 10 2025 was released in November 2025.

Key Web Threats we cover

- **SQL Injection (SQLi)**: Manipulates databases.
- **Cross-Site Scripting (XSS)**: Targets browsers.
- **XML External Entity (XXE)**: Exploits XML parsers.
- **Command Injection**: Runs OS commands.
- **File Traversal**: Accesses unintended files.
- **CSRF**: Forges user requests.

Why These Matter

- SQLi: Data theft (e.g., 2023 retail breach).
- XSS: Session hijacking—think banking apps.
- XXE: Server file leaks (e.g., /etc/passwd).
- File Traversal: Exposes system files.
- CSRF: Tricks users into actions (e.g., money transfers).

All preventable with the right approach.

SQL Injection: Basics

- Designed to exploit the nature of Web applications
- An SQL injection attack consists of insertion or “injection” of a SQL query via the input data from the client to the application
- **Goal**: Bypass auth, steal data.
- A successful SQL injection exploit can:
  - Read sensitive data from the database
  - Modify database data (Insert/Update/Delete)
  - Execute administration operations on the database (such as shutdown the DBMS)
  - Recover the content of a given file present on the DBMS file system
  - and in some cases issue commands to the operating system.
- How: Injects SQL via input (e.g., `' OR '1=1'`).

SQLi: Technique

- The SQLi attack typically works by prematurely terminating a text string and appending a new command
- Because the inserted command may have additional strings appended to it before it is executed the attacker terminates the injected string with a comment mark "- -"
  $\rightarrow$ Subsequent text is ignored at execution time

SQLi Attack Avenues

- **User input**: Attackers inject SQL commands by providing suitable crafted user input
- **Server variables** Attackers can forge the values that are placed in HTTP and network headers and exploit this vulnerability by placing data directly into the headers

Type of SQL Injections

Tautology

This form of attack injects code in one or more conditional statements so that they always evaluate to true

End-of-line comment

After injecting code into a particular field, legitimate code that follows are nullified through usage of end of line comments

Piggybacked queries

The attacker adds additional queries beyond the intended query, piggy-backing the attack on top of a legitimate request

Inferential Attack

There is no actual transfer of data, but the attacker is able to reconstruct the information by sending particular requests and observing the resulting behaviour of the Website/database server.

SQLi: How It Works

Login Authentication Query example

- Normal: `SELECT * FROM users WHERE user='admin' AND pwd='1234'`
- Injected: `SELECT * FROM users WHERE user='admin' OR '1=1' --'`
- -- skips the rest—auth bypassed.
  - Access obtained without password

User Authentication: Genuine vs. SQL Injection

Standard query to authenticate users:
`select * from users where user='$username' AND pwd='$password'`

Genuine Login

User Input: `user=admin, pass=1234`

Processed as

SQL Query:
`SELECT * FROM users WHERE user='admin' AND pwd='1234'`

Executes

Result: Access granted if valid

In Standard query to authenticate users:
In a genuine login attempt, the user provides a valid username and password. The server processes these inputs and constructs the SQL query.
If the credentials match an entry in the database, access is granted.
If not, the query returns an empty result, denying access.

SQL injection:
`select * from users where user='$username' AND pwd='$password'`

SQL Injection

User Input: `user=admin' OR '1=1, pass=xyz`

Injected into

SQL Query:
`SELECT * FROM users WHERE user='admin' OR '1=1' AND pwd='xyz' OR '1=1'`

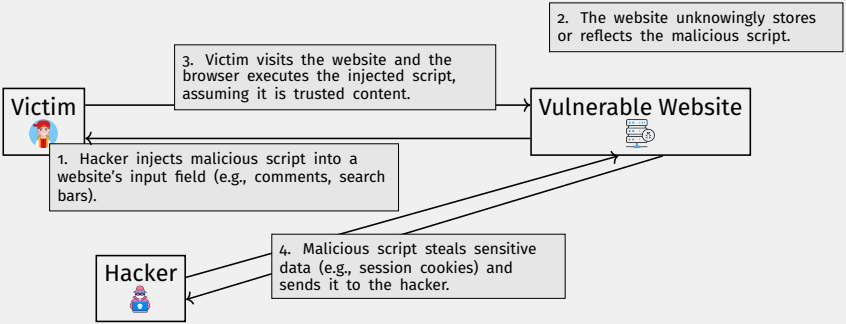
Executes

Result: Access granted without valid password

SQL Injection Explanation:
In a malicious login attempt, an attacker injects specially crafted SQL code into user input fields to manipulate database queries.
The server-side code retrieves the variables `$username` and `$passwd` from the web form and inserts them into the SQL query without proper sanitisation.
This allows the attacker to modify the logic of the query, bypassing authentication checks and gaining unauthorised access.

Cross Site Scripting (XSS) Attacks

- A cross-site scripting (XSS) attack is one that uses the web server to attack the client side.
- This injects a code fragment from a scripting language into an input field to have that code executed within the browser of a user visiting a site. For example <script></script> block



Cross Site Scripting (XSS) types

- There are three types of cross-site scripting attack. The difference is whether the script is stored somewhere or not.
  - **Persistent cross-site scripting**: Malicious script is stored on the server and automatically executed whenever users visit the affected page.
  - **Reflected cross-site scripting**: The script is not stored on the server. Instead, it is embedded in a URL and executed when the victim clicks the link.
  - **DOM-based XSS attack**: The attack manipulates the browser's Document Object Model (DOM), causing the browser to execute the malicious script when the page is rendered.

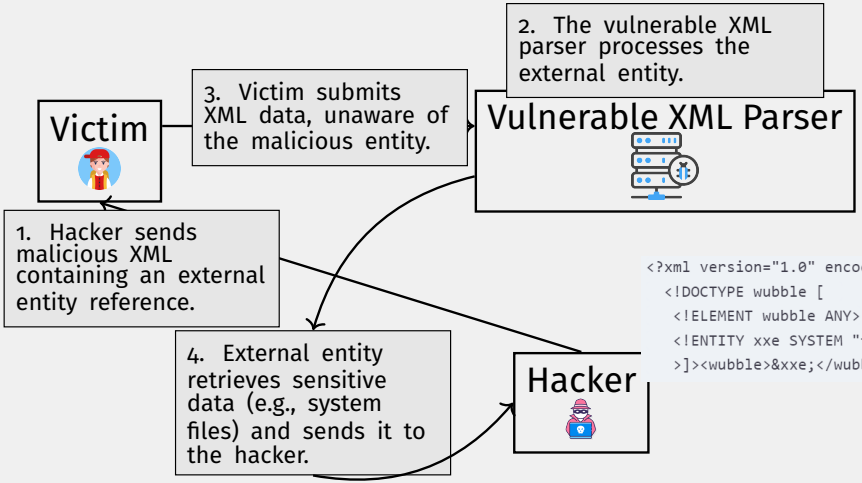
XML External Entity (XXE) attack

- A common way to pass data back and forth between a client and a server is to use XML to structure the data and transmit that XML.
- An XML entity injection attack comes from code on the web server accepting data that comes from the client without doing any data validation.
- We can even tamper with an existing XML page and parse different commands through the XML page.

XXE Attack

- Goal: Access server files.
- How: Exploits XML parsers with external entities.

XML External Entity (XXE) attack




```
<?xml version="1.0" encoding="ISO-8859-1"?>
  <!DOCTYPE wubble [
   <!ELEMENT wubble ANY>
   <!ENTITY xxe SYSTEM "file:///etc/passwd"
   >]><wubble>&xxe;</wubble>
```

Command Injection Attack

- Similar to an XML external entity injection attack.
- The application takes a value from the user and passes it to a system function or an evaluate function.
- Focus on the operating system
  - Pass the parameters to the operating system to handle.
- **Consequences**: If there is no input validation, you can execute any operating system command into the input field
- For example: If I enter ping -c 5 192.168.56.111 && cat /etc/passwd, the result will be the ping and the contents of the passwd file.

Command Injection attack

- **Goal**: Execute OS commands.
- **Example**: `ping && cat /etc/passwd`.

File Traversal Attack

- File traversal is a way to get out of what the web server wanted you to originally see, and be able to see more.

File Traversal Attack Basics

- Goal: Access files outside web root.
- How: Uses paths like `../../etc/passwd`.
- Risk: Exposes sensitive system files.

How It Works

- Normal: `/var/www/html/index.html`
- Injected: `/var/www/html/../../etc/passwd`
- Result: Server leaks file contents.

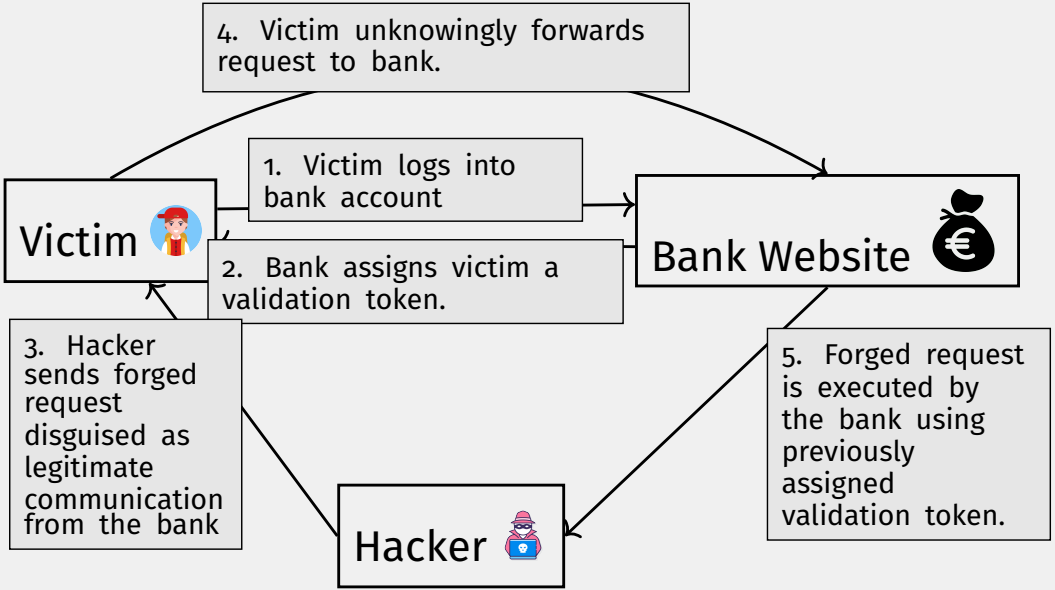
CSRF: Basics

- Cross-Site Request Forgery (CSRF) is a web security vulnerability that allows an attacker to induce a user to perform actions without their consent.
- Exploits the trust a website has in the user's browser.

CSRF: Basics

- **Goal**: Trick users into unwanted actions.
- **How**: Forges requests using victim's session.

CRSF EXAMPLE: HOW IT WORKS



Buffer Overflow Attack

- Programming error when a process attempts to store data beyond the limits of a fixed-sized buffer.
- A very common attack mechanism:
  - Overwrites adjacent memory locations can lead to:

    ↓↓↓

Basic Buffer overflow Example

- The attacker exploits an unchecked buffer to perform a buffer overflow attack
- The ultimate goal for the attacker is getting a shell that allows to execute arbitrary commands with high privileges

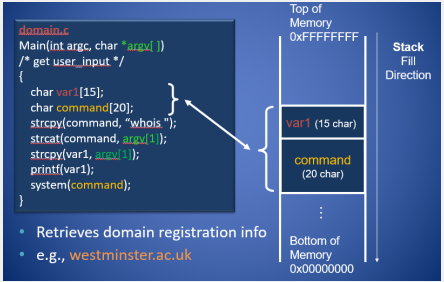


Figure 1: Memory contents Before the malicious input

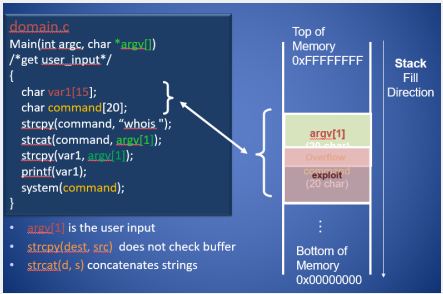


Figure 2: Memory contents after the malicious input, causing the Buffer Overflow

Overflow attack types

- Buffer Overflow in the stack:
  - This means that values of local variables, function arguments, and return addresses are affected.
  - Stack overflows corrupt memory on the stack.
- Buffer Overflow in the Heap:
  - Heap overflows refer to overflow that corrupts memory located on the heap (typically located above program code).
  - Memory is requested by programs to use in dynamic data structures (such as linked lists of records)
- Global variables and other program data are affected
  - A final category of buffer overflows involves buffers located in the program's global (or static) data area.
  - This is loaded from the program file and located in memory above the program code.

Reducing Software Vulnerabilities

- The NIST presents a range of approaches to reduce the number of software and web applications vulnerabilities
- It recommends:
  - Stopping vulnerabilities before they occur by using improved methods for specifying and building software and web applications
  - Finding vulnerabilities before they can be exploited by using better and more efficient testing techniques
  - Reducing the impact of vulnerabilities by building more resilient architectures

Writing Safe Program Code

- Second component is processing of data by some algorithm to solve required problem
- High-level languages are typically compiled and linked into machine code which is then directly executed by the target processor

Security issues

- Correct algorithm implementation
- Correct machine instructions for algorithm
- Valid manipulation of data

Correct Use of Memory

- Issue of dynamic memory allocation
  - Unknown amounts of data
  - Allocated when needed, released when done
  - Used to manipulate Memory leak
  - Steady reduction in memory available on the heap to the point where it is completely exhausted
- Many older languages have no explicit support for dynamic memory allocation
  - Use standard library routines to allocate and release memory
- Modern languages handle automatically

SQLi Countermeasures and prevention methods

Defensive coding

- Manual defensive coding practices
- Parametrised query insertion

Detection

- Signature based
- Anomaly based
- Code analysis

Run-time prevention

- Check queries at runtime to see if they conform to a model of expected queries

Buffer Overflow Defences Countermeasures and prevention

- Programmers often make assumptions about the maximum expected size of input and testing may not identify vulnerability
  - Allocated buffer size is not confirmed
  - Resulting in buffer overflow
- Buffer overflows are widely exploited.
- Safe coding treats all input as dangerous. Two broad defence approaches to do this:
- **Compile-time**
  - Aim to harden programs to resist attacks in new programs
- **Run-time**
  - Aim to detect and abort attacks in existing programs

Mitigating Web Threats

- **Validate Input**: Check size, type, source.
- **Parametrized Queries**: Stop SQLi.
- **Escape Output**: Block XSS.
- **Disable Entities**: Prevent XXE.

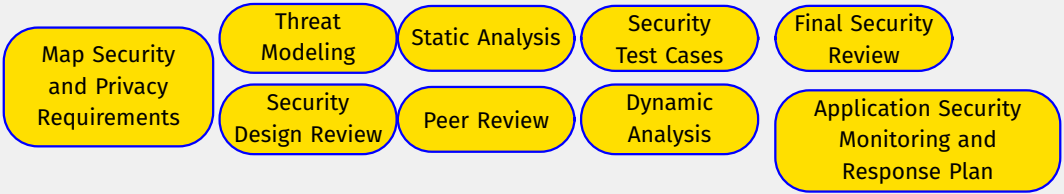
More Defences

- **Sanitise Paths**: Block File Traversal (e.g., reject `..`).
- **CSRF Tokens**: Unique codes per request.
- **Input Filtering**: Stop Command Injection.

Software Development Life Cycle

- Phases of the Software Development Life Cycle (SDLC)

- To make software secure, security must be built into the SDLC
- The earlier in the development life cycle security is implemented, the cheaper software development will be.



SDLC ACTIVITIES EXAMPLES

- **Requirements**: Define input and path rules.
- **Design**: Plan CSRF tokens, validation.
- **Coding**: Use safe APIs (e.g., PDO)
  - PHP Data Objects (PDO) is a PHP extension that provides a consistent, secure way to interact with databases (like MySQL, PostgreSQL, SQLite, etc.).
- **Testing**: Test traversal, forgery attempts.

References

- **Recommended Readings note:** Focus on what was covered in the class.
  - Chapter 9. **Web and Database Attacks**
    Hacker Techniques, Tools, and Incident Handling
  - Chapter 6. **Techniques Used by Hackers**
    Computer security fundamentals
- The lecture notes and contents were compiled from my own notes and from recommended books.
- Some of the Figures and tables are from the recommended books
- **You can use any of the recommended readings! You do not need to read all the chapters!**